



Pontificia Universidad
JAVERIANA
Bogotá

Proyecto 1 - Programación Avanzada 2026-30

Jimmy Alejandro Castro Sabogal

Proyecto: Juego de la mazmorra



Objetivos: Desarrollar un sistema en C++ que simula un videojuego en la consola de comandos.

Competencias esperadas: Apuntadores, estructuras, librerías, gestión de memoria dinámica, cadenas de caracteres, manejo de archivos.

Convenciones:

1. **PV:** Puntos de Vida
2. **PH:** Puntos de Habilidad

Problema

Como parte de un desafío de programación se le propone a su equipo el desarrollo de un videojuego tipo estrategia en consola, así como los desarrollados en tiempos anteriores, el cual consiste en escapar de una mazmorra la cual será representada como una matriz cuadrada de N filas y columnas y tendrá en ella una serie de desafíos por superar y una salida que estará en un punto aleatorio del borde del mapa.

Al principio del juego se contará con un personaje, el cual tendrá un nombre y unas estadísticas iniciales de vida (100PV) y habilidad (20PH) que se podrán mejorar, el personaje iniciará en un punto aleatorio del centro del tablero.

A lo largo del mapa se encontrarán una serie de cofres los cuales podrán contener pociones de vida, las cuales suman 50PV al personaje; bolsas de oro, los cuales entregan al usuario 10 piezas de oro; pero también podrá tratarse de cofres trampa, los cuales atacan al personaje y le restan 30 puntos de vida. Las probabilidades del cofre serán las siguientes: 50% de probabilidad de encontrar oro, 30% de encontrar una poción y 20% de ser un cofre trampa.

Además de las recompensas, dentro del mapa se encuentran una serie de enemigos los cuales estarán en lugares aleatorios del mapa y estarán dispuestos a acabar con el personaje, estos serán de 3 tipos distintos:

1. **Goblins:** Ataque físico, al estar en su casilla atacará. 30PV + 10PH.
2. **Arqueros:** Tiene la capacidad de atacar en las casillas adyacentes a su ubicación. 20PV + 10PH
3. **Jefe:** Ataque físico, más durabilidad y ataque, luego de 2 turnos el jefe se desplaza una casilla más cerca de la salida para cortarte el camino. Solo existe un jefe en el tablero. 60PV + 40PH.

Cuando el personaje vence a un enemigo se aumentarán los puntos de habilidad de la siguiente manera, Goblin (+2PH), Arquero (+3PH), Jefe (+5PH).

El tablero se compondrá de una serie de caracteres lo cuales representan elementos particulares descritos a continuación:

1. **Espacio vacío (' '):** Lugares vacíos de la mazmorra.
2. **Personaje ('@'):** Posición actual en la que se encuentra el personaje en el mapa
3. **Enemigo ('E'):** Posición de los enemigos, esto cambiará por el indicativo del enemigo al ser descubierto, Goblin ('G'), Arquero ('A'), Jefe ('J'). Al ser vencido cambia a ('X').

4. **Cofre ('?')**: Posición de los cofres, es necesario explorarlos, al ser abierto su carácter cambia a ('O')
5. **Salida ('#')**: Meta final del juego.

Con lo anterior un ejemplo de un tablero inicial, suponiendo un tamaño N de 10, con 3 enemigos y 3 cofres, del juego será el siguiente:

1		-	-	-	-	-	-	#	-	-	-	
2		-	E	-	-	-	-	E	-	-	-	
3		-	-	-	-	-	?	-	-	-	-	
4		-	-	-	-	-	-	-	-	-	-	
5		-	-	-	-	-	-	-	-	-	-	
6		-	-	?	-	-	@	-	-	-	-	
7		-	-	-	-	-	-	-	-	-	-	
8		-	-	-	-	-	-	-	-	-	-	
9		-	-	-	-	-	?	-	-	-	-	
10		-	-	E	-	-	-	-	-	-	-	

El juego

Estas serán las funciones que dictan el flujo normal del juego.

1. **Inicio del juego**: Al comenzar cada partida se deben ingresar las dimensiones del calabozo, los enemigos y los cofres, además de establecer de manera automática la salida y la posición inicial del jugador.
2. **Decisiones**: Una vez iniciado el juego, el personaje tendrá la opción de realizar uno de los siguientes movimientos:
 - a. **Explorar**: Abrir tesoros en la casilla actual.
 - b. **Moverse**: Desplazarse hacia alguna de las casillas adyacentes según la instrucción.
 - c. **Atacar**: Si hay un enemigo en la casilla, atacar al enemigo reduce sus puntos de vida.
3. **Actualización**: Luego de cada turno suceden siguientes eventos:
 - a. **Ataque de los arqueros**: Si el personaje está adyacente a un arquero se recibe un ataque y se informa al usuario.

- b. **Movimiento del jefe:** Cuando se cumple, el jefe se mueve a la siguiente casilla entre la salida y el jugador.
4. **Fin del juego:** Si el personaje llega hasta la salida o pierde todos sus puntos de vida se realiza el cálculo del puntaje final, este cálculo será almacenado en un reporte general el cual tendrá el nombre del personaje y el puntaje obtenido por este calculado de la siguiente manera:

$$P = B + (O * 2) + (PV * 2) + (PH * 5)$$

Donde:

- B: Puntaje base, si gana es de 100, sino será 0
- O: Cantidad de oro obtenido
- PV: Puntos de vida
- PH: Puntos de habilidad

Comandos del sistema

Para el desarrollo del sistema se solicita que se las instrucciones sean recibidas a través de la consola de comandos del sistema operativo, por lo tanto todas las instrucciones serán recibidas por el ejecutable del sistema e interpretadas en código para controlar el flujo según alguna de las siguientes opciones:

1. **Start:** Inicio del juego, en este punto se solicita al usuario las dimensiones de la matriz, la cantidad de cofres y la cantidad de enemigos de cada tipo y el sistema se encargará de crear el tablero.

```
PS D:\dev\C++\demo-proyecto> g++ .\main.cpp -o juego
PS D:\dev\C++\demo-proyecto> .\juego.exe start
N = 10
Goblins = 3
Arqueros = 2
Cofres = 3
Juego iniciado con exito

 1 | - - # - - - - - |
 2 | - - - - ? - - - - |
 3 | - - - - - - - E - - |
 4 | - E - - - - - - - |
 5 | - - - - @ - - - - - |
 6 | - - ? - - - - E - - - |
 7 | - - - - - - - - - - |
 8 | - - - - E - - - - - - |
 9 | - E - - - - ? - - - - |
10 | - - - - - - - - E - - |
```

2. **Board:** Obtener una vista del tablero

```
PS D:\dev\C++\demo-proyecto>
PS D:\dev\C++\demo-proyecto> .\juego.exe board
 1 | - - # - - - - - |
 2 | - - - - ? - - - - |
 3 | - - - - - - E - - |
 4 | - E - - - - - - - |
 5 | - - - - @ - - - - |
 6 | - - ? - - - - E - - |
 7 | - - - - - - - - - |
 8 | - - - - E - - - - |
 9 | - E - - - - ? - - - |
10 | - - - - - - - E - |
```

3. **Stats:** Ver las estadísticas actuales del personaje

```
PS D:\dev\C++\demo-proyecto> .\juego.exe stats
Stats:
PV:100
PH: 40
Botin: 100
```

4. **Move [up | down | left | right]:** Moverse sobre el mapa

```
PS D:\dev\C++\demo-proyecto> .\juego.exe move right
 1 | - - # - - - - - |
 2 | - - - - ? - - - - |
 3 | - - - - - - E - - |
 4 | - E - - - - - - - |
 5 | - - - - - @ - - - |
 6 | - - ? - - - - E - - |
 7 | - - - - - - - - - |
 8 | - - - - E - - - - |
 9 | - E - - - - ? - - - |
10 | - - - - - - - E - |
```

5. **Seek:** Explorar la casilla de tesoro, indica si es oro, poción o si es un cofre trampa
6. **Attack:** Hacer daño al enemigo que se encuentra en la misma casilla, indica si el enemigo es vencido con el ataque o no.

Para que haya una persistencia del estado del juego aun cuando el sistema solo permite ejecutar un comando a la vez, es necesario almacenar tanto las estadísticas del personaje como de los enemigos y el tablero como tal, para esto es requerido el uso de archivos binarios, ya que es un método eficiente para cargar y validar información.

Condiciones de entrega

Se debe entregar un archivo comprimido (.ZIP) con el código necesario para la ejecución del sistema además incluyendo los archivos de persistencia como caso de prueba.

El código debe contener tanto el archivo main.cpp como los archivos header (.h y .hxx) con las estructuras y funciones que fueron usadas en el desarrollo del proyecto, debe existir al menos un archivo de header para asegurar el orden del código.